

# INFRATHRONE COREOPS PLAYBOOK

## A Troubleshooting Manual for Early-Stage DevOps & SRE Engineers

Week-1 → CoreOps Track

---

### Purpose of This Playbook

This playbook teaches you:

- How to think like a real production engineer
  - What to check first when an issue hits
  - How to avoid panic and build a stable debugging sequence
  - How to see signals in the noise
  - How to reduce MTTR by following a consistent mental model
- 

**Golden Rule of CoreOps Debugging:** Never touch the system before you understand the symptom.

Every production failure begins with **one of these 5 signals**:

1. **Performance degradation**
2. **Service unreachable**
3. **Pod/application crash**
4. **Resource pressure**
5. **Networking failure**

**Before running commands, always ask:** “What EXACTLY is failing? Component, symptom, and impact.”

---

## CoreOps Debugging Framework (The InfraThrone 6-Step Loop)

This framework is *mandatory* for all CoreOps engineers:

---

### STEP 1: Identify the Symptom

Ask the first three questions:

1. *What is the user complaining about?*
  2. *What is the exact error message or behavior?*
  3. *Is this affecting one pod, one node, one service, or the entire cluster?*
- 

### STEP 2: Check Pod / App Status

Run these non-harmful commands:



```
kubectl get pods -A  
kubectl get pods -n <namespace>  
kubectl describe pod <name>  
kubectl logs <name>  
kubectl get events --sort-by=.metadata.creationTimestamp
```

Focus only on:

- Restarts
  - CrashLoopBackOff
  - ImagePullBackOff
  - Liveness/readiness issues
  - Event warnings
-

### STEP 3: Check Node & Resource Health

Basic node health check:

```
kubectl get nodes
kubectl describe node <node>
kubectl top nodes
kubectl top pod -A
```

Look for:

- CPU starvation
- Memory pressure
- Disk pressure
- NetworkUnavailable
- Kubelet issues

These signals account for **60% of early-career outages**.

---

### STEP 4: Check Service & Networking Path

Networking issues often *look* like application issues.

Run:

```
kubectl get svc -A
kubectl get ep -A
kubectl exec -it <pod> -- curl http://<service>:<port>
nslookup <service-name>
```

Check:

- Endpoints missing?
  - DNS resolving correctly?
  - Traffic reaching the correct pod?
-

## STEP 5: Validate the Configuration

Check the YAML written vs. the YAML applied:

```
kubectl get deploy <name> -o yaml
kubectl describe deploy <name>
kubectl get configmap <name> -o yaml
kubectl get secret <name> -o yaml
```

Look for:

- Wrong port numbers
- Wrong env vars
- Wrong image tags
- Wrong probes
- Wrong resource requests

These account for **85% of CoreOps mistakes**.

---

## STEP 6: Fix, Validate, Prevent

Every fix **MUST** be followed by:

```
kubectl rollout status deploy/<name>
kubectl get pods -n <namespace>
```

And prevention steps:

- Add proper probes
  - Add resources
  - Create runbooks
  - Document the RCA
-

## Most Common Outage Patterns (CoreOps Level)

### POD-LEVEL

1. CrashLoopBackOff
2. ImagePullBackOff
3. OOMKilled
4. Readiness/Liveness probe failures

### NODE-LEVEL

5. Node memory pressure
6. Disk full
7. Node not ready
8. Kubelet CPU starvation

### NETWORK-LEVEL

9. Broken service endpoints
10. DNS failures
11. Pod cannot reach another pod
12. Port mismatch

Every pattern above can be solved using the CoreOps 6-Step Loop.

---

## CoreOps Debugging Commands: The Essential Set

Your **daily use** commands:

### Pod lifecycle

```
kubectl get pods -A  
kubectl describe pod <name>  
kubectl logs <name>  
kubectl logs -f <name>
```

### Events

```
kubectl get events --sort-by=.metadata.creationTimestamp
```

### Resource pressure

```
kubectl top pods -A  
kubectl top nodes
```

### Networking

```
kubectl get svc  
kubectl get ep  
kubectl exec -it <pod> -- curl <service>:<port>  
nslookup <service>
```

### Configuration

```
kubectl get deploy <name> -o yaml  
kubectl describe deploy <name>
```

### Node health

```
kubectl describe node <node>  
journalctl -u kubelet (if you have SSH)
```

INFRATHRONE

THE DEVOPS WAR ROOM

## How to Log Your Debugging (InfraThrone Standard)

Every engineer must maintain this structure in notes:

1. **Symptom**
2. **Initial hypothesis**
3. **Commands used**
4. **Clues found**
5. **Root cause**
6. **Fix applied**
7. **Prevention steps**

This trains your mind to think like a **war-room engineer**.

---

## Mini Labs You Must Practice (CoreOps Level)

These small labs **MUST** be completed in Week-1:

### Scenario A: Pod stuck in “ContainerCreating”

Investigate:

- events
- node volumes
- image pull

## Scenario B: CrashLoopBackOff

Investigate:

- logs
- probes
- env vars

## Scenario C: Service unreachable

Investigate:

- endpoints
- DNS
- pod health

## Scenario D: High node CPU

Investigate:

- top nodes
- pods using CPU
- kubelet logs

These labs give you **real outage muscle memory**.

---



## CoreOps Mindset Rules

1. Observe before you act.
2. Collect clues systematically.
3. One hypothesis at a time.
4. Don't chase 20 things at once.
5. Never skip events.
6. Never ignore node health.
7. Never assume the YAML is correct.
8. Write your learning memory compounds.

---

## Final Message from InfraThrone

CoreOps is not “basic DevOps.”

CoreOps is the **foundation of war-room mastery**.

A weak foundation → panic in outages.

A strong foundation → calm, precise, reliable execution.

This playbook ensures you never fear the terminal, the cluster, or the outage.

You are building your **firefighting instincts** — one command at a time.